大模型本地部署

LM Studio

https://lmstudio.ai/

LM Studio是一个简单易用的桌面端应用程序，通过简单的安装配置，我们可以轻松通过该软件在自己本地的电脑上部署大模型使用

在安装的时候，最好启用开发者模式，这样方便我们后续可以查看和调整一些高级配置

除了提供模型的下载和访问之外，还提供了API Server的工作模式，可以将本机部署的大模型通过标准化的API提供给其他机器和应用程序进行调用

HuggingFace

https://huggingface.co

HuggingFace是目前全球最大的开源模型和数据集平台，在该平台上可以获取到所有的开源AI大模型

不过国内的网络现在已经无法直接访问HuggingFace

ModelScope

对于我们在国内使用的情况下，使用ModelScope是很好的替代选择，这是国内与阿里云平台合作的HuggingFace的平替平台，并且方便国内的网络使用

下载模型

我们这里以Qwen3.5-4B模型举例，这是一个4B参数的模型，大约8GiB多的大小

https://www.modelscope.cn/models/Qwen/Qwen3.5-4B

在我们的Linux系统环境中，Python不是一个简单的用于开发的软件环境或者运行时，Python已经成为了系统中很多重要基础功能的核心组件

由于Python这个编程语言诞生比较早，并且设计上存在缺陷，我们现在推荐使用python中的`venv`功能，创建一个独立的干净的python环境，与系统级的环境进行隔离，这样我们进行任何操作，都不会影响系统级的python环境

我们在Rocky Linux上需要确保`python3-devel`软件包已经安装

```
dnf install python3-devel
```

我们创建python虚拟环境并激活该环境

```
python3 -m venv .venv
# 使用python内的venv模块，在当前目录位置创建.venv目录
# 这里的.venv目录就是我们的虚拟python环境

source .venv/bin/activate
# 使用source命令，通过提供的脚本文件，激活我们的虚拟环境
# 该脚本文件本质上是通过修改相关的环境变量来进行隔离的实现
# 需要注意，我们如果重新启动了终端或者重新登陆之后，
# 都需要手动的重新加载一个这个脚本才行
```

如果我们成功加载了虚拟环境，我们的命令行会有如下的特点

```
((.venv) ) [root@localhost ~]#
```

这时候，我们的命令行前面会有小括号，里面是我们虚拟环境的对应目录位置，用于标识我们现在所有有关python的操作之影响我们虚拟环境

接下来，我们可以直接根据官方提供的流程，下载我们的模型权重

```
pip install modelscope
```

如果相关下载较慢，可以先配置国内的镜像仓库

https://mirrors.cernet.edu.cn/

https://help.mirrors.cernet.edu.cn/pypi/

我们就可以通过一条命令，来下载我们的模型原始文件

```
modelscope download --model Qwen/Qwen3.5-4B  --local_dir ./Qwen3.5-4B
```

当我们有了模型的原始权重文件之后，我们需要让我们的模型跑起来，最传统的方式就是用python中的`pytorch`等框架，反向的加载我们这里的模型权重，一定程度上还原到训练大模型的时候的状态，通过代码来完成相关的调用

我们需要借助一个项目，该项目实现了用纯CPU的方式也能高效的运行我们的大模型（推理），而且不需要太多的软件依赖，并且支持多种不同的硬件设备实现加速，或者多卡多设备多机推理

llama.cpp

https://github.com/ggml-org/llama.cpp

我们首先需要从Github上面下载源码，一般来说，我们使用`git clone`的方式来将源代码克隆一份到本地

我们需要确保我们安装了`Git`

```
git clone --depth 1 https://github.com/ggml-org/llama.cpp.git
# 我们这里使用--depth 1表示我们只克隆最新版本的代码
# 如果不加--depth 1，那么就会把这个仓库代码的所有历史版本全部完整的克隆下来
```

由于GitHub在国外，国内在clone代码的时候可能不稳定，所以我这里提供一个源码的备用下载地址

```
curl https://file.boulderh.top/llama.cpp.tar -o ./llama.cpp.tar
# 下载

tar -xf llama.cpp.tar
# 从归档文件中提取
```

我们需要使用源码完成两个操作

1. 使用里面的python脚本，将我们的原始的模型权重转换为专用的GGUF格式
2. 使用GCC和cmake等工具，编译我们的项目，生成我们的`llama.cpp`相关的可执行文件用于执行，量化和将模型作为服务进行提供

```
pip install -r llama.cpp/requirements.txt
# 安装里面相关python脚本的依赖
```

安装好依赖之后，我们直接调用源码中的脚本进行转换

```
python llama.cpp/convert_hf_to_gguf.py ./Qwen3.5-4B
# 调用专门的转换脚本对原始的模型权重转换为gguf文件
# 并且这个新生成的gguf文件会保存在./Qwen3.5-4B目录内
```

我们的GGUF文件的大小，一定程度上对应了我们需要的显存+内存的大小，如果我们空间不足，是没法运行我们的模型的（会触发操作系统的保护，直接kill我们的模型）

我们需要使用llama.cpp里面的专用工具，来对模型进行量化处理，量化是一种特殊技术，将模型原本从半精度（BF16）转换为低精度（Q8_0，Q4_K_M）等，减少了模型原本的大小和推理压力，但是会损失模型的能力（模型降智），一般来说，更大参数的模型，在使用了量化之后通常依旧比同大小的高精度小参数的模型能力更强

我们需要编译我们的`llama.cpp`之后我们才有专门的量化工具

我们需要先安装一些编译时候使用的工具链

```
dnf group install "Development Tools" -y

dnf install ccache cmake openssl-devel nodejs -y
# 安装llama.cpp项目中一些额外的编译组件
# cmake是必须要安装的，用于生成编译脚本进行gcc等指令的操作
# ccache是可选的（需要启用epel-release），主要用于多次编译的时候能够减少不必要的编译内容
# openssl-devel是可选的，主要让我们的llama.cpp支持例如https等安全协议
# nodejs是可选的，主要是编译llama-server的前端web界面的支持
```

这里的nodejs我们需要单独配置一下国内的镜像仓库

https://developer.aliyun.com/mirror/

https://www.npmmirror.com/

```
npm config set registry https://registry.npmmirror.com
```

我们开始进行编译操作（新版本下，我们编译之前需要确保虚拟机内存提升到8G）

```
cd llama.cpp/
# 将当前目录切换到llama.cpp源码目录

cmake -B build
# 生成编译脚本，这里会同步检查我们的环境是否满足最基本的编译要求
# 如果我们的系统存在任何会导致编译无法正常进行的情况，这里会无法正常生成编译目录
# 有一些编译的选项或者环境是可选的，这里会提示我们


cd ~
cd llama.cpp/tools/ui
# 我们可以先单独的编译一下前端页面
npm install
# 安装nodejs依赖
npm run build
# 单独对前端页面进行编译
# 如果我们虚拟机内存足够，可能能够正常的进行这里的编译


cd ~
cd llama.cpp
# 回到我们的llama.cpp源码目录
cmake --build build --config Release
# 开始进行编译
```

编译相对需要的时间比较长，我们需要相对比较长的时间等待

等待编译完成之后，我们的在`build`目录下的`bin`目录中就可以有大量的可执行的命令来进行一些操作

```
cd ~

ls llama.cpp/build/bin
# 我们编译好之后的所有产物，都在这个目录内
```

量化我们使用`llama-quantize`进行模型的量化操作

```
./llama.cpp/build/bin/llama-quantize Qwen3.5-4B/Qwen3.5-4B-BF16.gguf Qwen3.5-4B/Qwen3.5-4B-Q4_K_M.gguf Q4_K_M
# 将我们之前转换得到的BF16精度的模型直接转换为Q4_K_M精度的模型
# 转换之后的文件名为Qwen3.5-4B-Q4_K_M.gguf
```

原本`8253.72 MiB`模型，经过我们这里的量化，会变成`2644.04 MiB`

这之后，我们就可以使用`llama-cli`命令直接在命令行中加载我们的模型，在命令行上和我们的模型进行交互

```
./llama.cpp/build/bin/llama-cli -m Qwen3.5-4B/Qwen3.5-4B-Q4_K_M.gguf
```

我们也可以将其转为web服务的方式进行提供

```
systemctl disable --now firewalld
# 我们这里先简化一下，先关闭防火墙服务

./llama.cpp/build/bin/llama-server -m Qwen3.5-4B/Qwen3.5-4B-Q4_K_M.gguf --host 0.0.0.0 --port 8080 -a Qwen3.5-4B
# 使用llama-server启动openai兼容服务器，使用Qwen3.5-4B/Qwen3.5-4B-Q4_K_M.gguf作为模型文件
# --host 0.0.0.0对外提供服务（监听所有的IP地址）
# --port 8080 指定我们的访问端口为8080端口
# -a Qwen3.5-4B 指定我们对外的时候模型的名字
```

通过浏览器访问，我们就可以跟模型进行交互

我们在转换模型为GGUF格式的时候，对于有视觉能力的模型，我们还需要单独转换视觉投影器

```
python llama.cpp/convert_hf_to_gguf.py ./Qwen3.5-4B --mmproj
# 单独转换我们模型的视觉投影部分
# 我们会得到mmproj-Qwen3.5-4b-BF16.gguf这样的一个投影文件
```

这里我们有可能会遇上我们Linux系统存储空间不足的问题（`df -Th`命令中可以看到`/`目录基本用满了）

我们的方法可以删除`BF16`精度GGUF文件删除，或者卸载我们之前为了编译安装的相关的一些软件，腾出我们的空间

当然我们也可以给虚拟机添加额外的硬盘，通过`lvm`给虚拟机的根目录进行扩容操作

我们如果想要支持图片理解层面的支持，命令行中需要增加`-mm`来指定我们的投影文件

```
./llama.cpp/build/bin/llama-server -m Qwen3.5-4B/Qwen3.5-4B-Q4_K_M.gguf -mm Qwen3.5-4B/mmproj-Qwen3.5-4b-BF16.gguf --host 0.0.0.0 --port 8080 -a Qwen3.5-4B
```